

COMP1324 Algorithmics

Dynamic Programming 3

Hikmat Farhat

Rod Cutting

- A steel company buys rods and cuts them into smaller pieces
- The company sells rods of different (integer) lengths, i
- Each particular length, i , has a different price $p[i]$
- Given a rod of length n the company would like to cut it into smaller pieces to maximize the total revenue.
- Example prices for a rod of length 4 are shown below

i	1	2	3	4
$p[i]$	1	5	8	9

- Let x_i denote whether the rod is cut at position i
- Each x_i has two possibilities: 0 (no cut) or 1 (cut)
- Since $1 \leq i \leq n - 1$ then there are 2^{n-1} possibilities

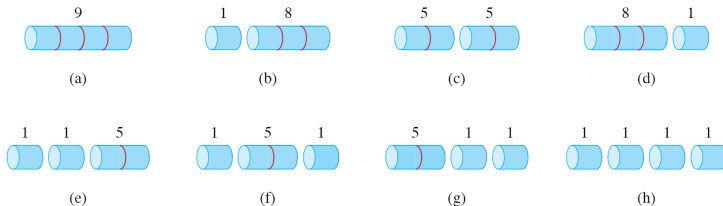
Example: 8 possibilities for $n = 4$ 

Figure: Different ways of cutting a rod of length 4.(CLRS)

- Maximum revenue is obtained using the cut $p[2] + p[2]$ for a total of 10 (part (c) in the figure)

Recursive Solution

- Let $r(n)$ be the maximum revenue obtainable from a rod of length n
- When writing the solution in terms of the solution to subproblems we have to make sure what is exactly the subproblem.
- In the expression for $r(n)$ it is **implicitly** assumed that we can cut it at any position i , $1 \leq i \leq n - 1$
- To make that dependence **explicit**, denote by c_i the set of variables $\{x_1 \dots x_i\}$ where $x_i \in \{0, 1\}$.
- Then $r(n) \equiv r(c_{n-1}, n)$, meaning a rod of length n which can be cut at any position i , $1 \leq i \leq n - 1$
- Consider a rod of length n and the two possibilities: it is either cut or not cut at position x_{n-1} we can write

$$r(c_{n-1}, n) = \max(r(c_{n-2}, n), r(c_0, 1) + r(c_{n-2}, n - 1))$$

Recursive Solution

- Notice that $r(c_{n-2}, n) \not\equiv r(n-1) \equiv r(c_{n-2}, n-1)$, i.e. it is **not a subproblem** of $r(n)$
- Similarly, a rod of length n can be cut at x_{n-2} or not. Then

$$r(c_{n-2}, n) = \max(r(c_{n-3}, n), r(c_1, 2) + r(c_{n-3}, n-2))$$

- Since $\max(\max(a, b), c) = \max(a, b, c)$ we can write

$$r(c_{n-1}, n) = \max(r(c_{n-3}, n), r(c_1, 2) + r(c_{n-3}, n-2), \\ r(c_0, 1) + r(c_{n-2}, n-1))$$

- Dropping the c's and taking into account that $r(c_0, n) = p[n]$ the above can be generalized to

$$r(n) = \max(p[n], r(n-1) + r(1), \dots$$

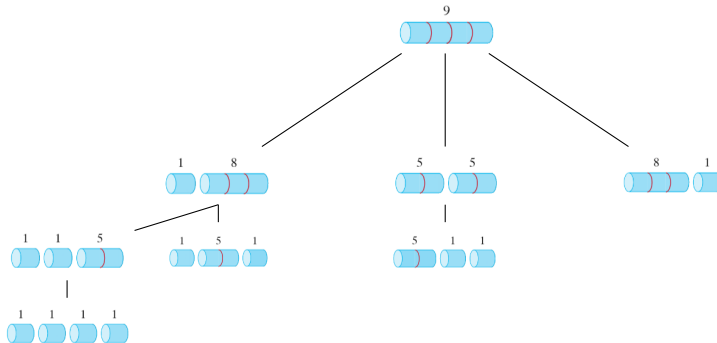
$$r(2) + r(n-2), r(1) + r(n-1))$$

Simplifying the recursion

- Suppose that we cut a rod of length n into two pieces of length i and $n - i$.
- That particular cut will give us a revenue of $r(i) + r(n - i)$
- We recurse on the left ($r(i)$) and right branch $r(n - i)$
- We will simplify the recursive relation by using a single recursive call instead of two.

One branch can generate all possibilities

- No need to recurse on both branches because they duplicate the same possibilities
- For example, below is the recursion tree on the right branch for a rod of length 4



Recursive solution

The simplified recurrence becomes

$$r(n) = \max_{k=1}^n [p(k) + r(n - k)]$$

It can be implemented using the following algorithm

Algorithm 1: Recursive algorithm for rod cutting

function CUT-ROD(p, n)

if $n = 0$ **then**

return 0

$r \leftarrow -\infty$

for $i \leftarrow 1$ **to** n **do**

$r \leftarrow \max(r, p[i] + \text{CUT-ROD}(p, n - i))$

return r

Complexity

- The complexity of the above obeys

$$\begin{aligned}T(n) &= 1 + \sum_{i=0}^{n-1} T(i) \\&= \left(1 + \sum_{i=0}^{n-2} T(i)\right) + T(n-1) \\&= 2T(n-1)\end{aligned}$$

- Iterating the above recurrence with $T(0) = 1$ we find

$$T(n) = 2^n$$

Overlapping subproblems

- The exponential blowup is due to the repeated computation of subproblems
- The recursion tree for the case $n=4$ is show below

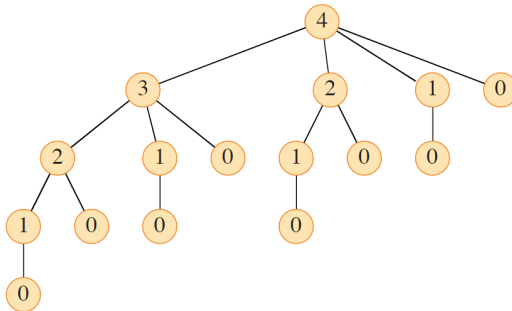


Figure: Recursion tree for $n=4$ (CLRS)

Memoization

Algorithm 2: Memoized algorithm for rod cutting

for $i \leftarrow 1$ **to** n **do**

$r[i] \leftarrow -\infty$

function MEMO-CUT-ROD(p, n)

if $r[n] > 0$ **then**

return $r[n]$

if $n = 0$ **then**

return 0

$m \leftarrow -\infty$

for $i \leftarrow 1$ **to** n **do**

$m \leftarrow \max(m, p[i] + \text{MEMO-CUT-ROD}(p, n - i))$

$r[n] \leftarrow m$

return m

Bottom up solution

- The easiest way to implement DP is by using a bottom-up approach.
- Basically, the smaller subproblems are computed first.

Algorithm 3: Bottom up algorithm for rod cutting

function BOTTOM-UP-CUT-ROD(p, n)

```
     $r[0] \leftarrow 0$   
    for  $j \leftarrow 1$  to  $n$  do  
         $m \leftarrow -\infty$   
        for  $i \leftarrow 1$  to  $j$  do  
             $m \leftarrow \max(m, p[i] + r[j - i])$   
         $r[j] \leftarrow m$   
    return  $r[n]$ 
```

- Complexity?

Bottom up solution

- The easiest way to implement DP is by using a bottom-up approach.
- Basically, the smaller subproblems are computed first.

Algorithm 4: Bottom up algorithm for rod cutting

function BOTTOM-UP-CUT-ROD(p, n)

```
     $r[0] \leftarrow 0$   
    for  $j \leftarrow 1$  to  $n$  do  
         $m \leftarrow -\infty$   
        for  $i \leftarrow 1$  to  $j$  do  
             $m \leftarrow \max(m, p[i] + r[j - i])$   
         $r[j] \leftarrow m$   
    return  $r[n]$ 
```

- Complexity? $\Theta(n^2)$

Bottom up solution

- The easiest way to implement DP is by using a bottom-up approach.
- Basically, the smaller subproblems are computed first.

Algorithm 5: Bottom up algorithm for rod cutting

function BOTTOM-UP-CUT-ROD(p, n)

```
     $r[0] \leftarrow 0$   
    for  $j \leftarrow 1$  to  $n$  do  
         $m \leftarrow -\infty$   
        for  $i \leftarrow 1$  to  $j$  do  
             $m \leftarrow \max(m, p[i] + r[j - i])$   
         $r[j] \leftarrow m$   
    return  $r[n]$ 
```

- Complexity? $\Theta(n^2)$ (why?)

Max subarray sum (leetcode #53)

Given an array of numbers (with at least one negative number) $A_0, A_1, \dots, A_{n-1}$ find subarray $B = [A[i], \dots, A[j]]$ whose sum of elements is maximum. In other words let

$$S_{ij} = \sum_{k=i}^{j \geq i} A_k$$

Goal: find

$$\max_{ij} S_{ij}$$

- Brute force: $\Theta(n^3)$
- Divide and conquer: $\Theta(n \log n)$
- Can we do better? yes

Example

- Consider the array

$$A = [2, -4, 6, 3, -7, 4, 5, -5, -6, 4, 6, -4, 3]$$

- One can guess by inspection that the subarray $[6, 3, -7, 4, 5]$ will give the largest sum (11).
- Define the max partial suffix $M_j = \max_i S_{ij}$.
- Then we can write

$$\max_{ij} S_{ij} = \max_j M_j$$

- If we compute M_j for all j then we can find the maximum subarray sum.

Partial sums recurrence

$$M_j = \max \left\{ \begin{array}{l} A[1] + A[2] + A[3] + \dots + A[k] + \dots + A[j] \\ A[2] + A[3] + \dots + A[k] + \dots + A[j] \\ A[3] + \dots + A[k] + \dots + A[j] \\ \dots \\ A[k] + \dots + A[j] \\ \dots \\ A[j] \end{array} \right.$$

Partial sums recurrence

$$M_{j+1} = \max \left\{ \begin{array}{l} A[1] + A[2] + A[3] + \dots + A[k] + \dots + A[j] + A[j+1] \\ A[2] + A[3] + \dots + A[k] + \dots + A[j] + A[j+1] \\ A[3] + \dots + A[k] + \dots + A[j] + A[j+1] \\ \dots \\ A[k] + \dots + A[j] + A[j+1] \\ \dots \\ A[j] + A[j+1] \\ A[j+1] \end{array} \right.$$

$$= \max(M_j + A[j+1], A[j+1])$$

Where we have used

$$\max(x_1 + a, x_2 + a, \dots, x_n + a) = \max(x_1, x_2, \dots, x_n) + a$$

Solving the original problem

- We have obtained a recurrence for M_j .
- This allows us to compute M_j for all j in $\Theta(n)$ time.
- The maximum subarray sum is then $\max_j M_j$.
- Consider again the array

$$A = [2, -4, 6, 3, -7, 4, 5, -5, -6, 4, 6, -4, 3]$$

- using the proposed algorithm we get

$$M = [2, -2, 6, 9, 2, 6, 11, 6, 0, 4, 10, 6, 9]$$

- By scanning the result we obtain the maximum 11 as it should be.
- Note that this method not only computes the result for $n = 13$ elements but also solves **all subproblems**

Pseudo code

```
function MaxSubarraySum(A)  
     $M[0] \leftarrow 0$   
    for  $i = 1$  to  $n$  do  
        |  $M[i] \leftarrow \max(A[i], A[i] + M[i - 1])$   
    /* Now compute the maximum of all sums          */  
     $m \leftarrow 0$   
    for  $i = 1$  to  $n$  do  
        | if  $M[i] > m$  then  
        | |  $m \leftarrow M[i]$   
        | |  $idx \leftarrow i$   
    return  $m, idx$ 
```

- Time complexity?
- Space complexity?

Pseudo code

```
function MaxSubarraySum(A)  
     $M[0] \leftarrow 0$   
    for  $i = 1$  to  $n$  do  
         $M[i] \leftarrow \max(A[i], A[i] + M[i - 1])$   
    /* Now compute the maximum of all sums          */  
     $m \leftarrow 0$   
    for  $i = 1$  to  $n$  do  
        if  $M[i] > m$  then  
             $m \leftarrow M[i]$   
             $idx \leftarrow i$   
    return  $m, idx$ 
```

- Time complexity? $\Theta(n)$
- Space complexity?

Pseudo code

```
function MaxSubarraySum(A)  
    M[0]  $\leftarrow$  0  
    for i = 1 to n do  
        | M[i]  $\leftarrow$  max(A[i], A[i] + M[i - 1])  
    /* Now compute the maximum of all sums          */  
    m  $\leftarrow$  0  
    for i = 1 to n do  
        | if M[i] > m then  
        | |   m  $\leftarrow$  M[i]  
        | |   idx  $\leftarrow$  i  
    return m, idx
```

- Time complexity? $\Theta(n)$
- Space complexity? $\Theta(n)$

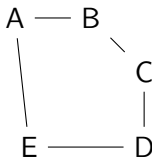
Recovering the solution

- To determine i, j such that S_{ij} is maximum we first note that j is the index of the maximum value in the M array.
- We then scan backwards from j to find i such that $M[i] = A[i]$.
- In the example we had

$$A = [2, -4, 6, 3, -7, 4, 5, -5, -6, 4, 6, -4, 3]$$
$$M = [2, -2, 6, 9, 2, 6, 11, 6, 0, 4, 10, 6, 9]$$

Don't all problem have optimal substructure?

- No. For example, the longest path problem does not have the optimal substructure
- The longest (simple) path from A to D cannot be constructed from longest paths $B - D$, $C - D$ or $E - D$



References

- The rod cutting problem is covered in section 14.1 of your [textbook](#)
- A good discussion of optimal substructure can also be found in your textbook section 14.3
- A Python implementation of the maximum subarray sum problem can be found [here](#)